Studio Nx

Backend(FastAPI/Python) 10 — Project 모델 컬럼

2026-10-06

Backend(FastAPI/Python) 10 — `Project` 모델 컬럼

항목	내용
파일	`studionx/backend/app/models/project.py` (`Project` 클래스)
작업	P1-03 백엔드 공통 — ORM 모델 컬럼 · `docs/DB.md` §3.1 · `studionx/db/01_phase1_account.sql`
작성	Claude (AI) · 2026-10-06 · 사용자 검토
이전 · 다음	09 `Inquiry` 모델 · 11 `ProjectMember` 모델

1. 하는 일

MySQL 의 `projects` 테이블(시퀀서 프로젝트 한 개)을 파이썬 클래스 `Project` 로 옮겨 적는다. 08 · 09 와 같은 방식이고, 이번에 새로 나온 타입은 셋이다.

새로 나온 타입	칸	파이썬에서 읽으면
`DECIMAL(6,2)` (정확한 소수)	`bpm`	`Decimal("120.00")`
`TINYINT UNSIGNED` (0 ~ 255)	`time_sig_num` · `time_sig_den` (박자 4/4)	`4`
`JSON`	`master_settings` (마스터 채널 설정)	사전 `{"gain_db": -1.5}`

같은 파일의 `ProjectMember` 는 11, 관계(`project.members` · `project.tracks`)는 `ProjectMember` · `Track` 모델을 채운 뒤에 붙인다.

2. 언제 쓰나

쓰는 곳	화면	칸
P1-13 프로젝트 목록 · 만들기 · 이름 바꾸기 · 삭제	myProject.html	`owner_id` · `title` · `description` · `updated_at` · `deleted_at`
P1-16 대시보드 최근 프로젝트	myPage.html	`title` · `updated_at` · `thumbnail_path`
Phase 2 시퀀서	studioNx.html 위쪽 막대 (BPM · 박자 · 키)	`bpm` · `time_sig_*` · `music_key` · `scale_mode` · `sample_rate` · `master_settings` · `revision`

3. 코드

import (파일 맨 위)

```
from datetime import datetime
from decimal import Decimal
from typing import Any

from sqlalchemy import JSON, BigInteger, FetchedValue, ForeignKey, Integer, Numeric, String, text
from sqlalchemy.dialects.mysql import DATETIME, TINYINT
from sqlalchemy.orm import Mapped, mapped_column

from app.db.session import Base
```

`BigInteger` · `ForeignKey` · `Mapped` · `mapped_column` · `Base` 는 원래 있었고, 나머지를 이번에 추가했다.
`ProjectMember`(11)도 같은 import 를 쓴다.

import	가져오는 것	쓰는 곳
`Decimal` (파이썬 기본)	정확한 소수	`bpm`
`Any` (파이썬 기본)	"아무 타입" 표시	`dict[str, Any]` — JSON 안의 값
`Numeric(6, 2)`	SQL 의 `DECIMAL(6,2)`	`bpm`
`TINYINT` (MySQL 전용)	1바이트 정수	박자
`Integer`	SQL 의 `INT`	`sample_rate` · `revision`
`JSON`	SQL 의 `JSON`	`master_settings`
나머지	08 의 표 와 같음	

함수 (클래스 · 칸 선언)

```
class Project(Base):
    __tablename__ = "projects"

    project_id: Mapped[int] = mapped_column(BigInteger, primary_key=True, autoincrement=True)

    owner_id: Mapped[int] = mapped_column(BigInteger, ForeignKey("users.user_id"))
    title: Mapped[str] = mapped_column(String(100))
    description: Mapped[str | None] = mapped_column(String(500))
    bpm: Mapped[Decimal] = mapped_column(Numeric(6, 2), server_default=text("120.00"))
    time_sig_num: Mapped[int] = mapped_column(TINYINT(unsigned=True), server_default=text("4"))
    time_sig_den: Mapped[int] = mapped_column(TINYINT(unsigned=True), server_default=text("4"))
    music_key: Mapped[str | None] = mapped_column(String(10))
    scale_mode: Mapped[str | None] = mapped_column(String(20))
    sample_rate: Mapped[int] = mapped_column(Integer, server_default=text("48000"))
    master_settings: Mapped[dict[str, Any] | None] = mapped_column(JSON)
    revision: Mapped[int] = mapped_column(Integer, server_default=text("0"))
    thumbnail_path: Mapped[str | None] = mapped_column(String(500))
    # FK → project_templates (Phase 2). 그 모델이 생기면 ForeignKey 를 붙인다.
    template_id: Mapped[int | None] = mapped_column(BigInteger)
    created_at: Mapped[datetime] = mapped_column(
        DATETIME(fsp=3), server_default=text("CURRENT_TIMESTAMP(3)")
    )
    updated_at: Mapped[datetime] = mapped_column(
        DATETIME(fsp=3), server_default=text("CURRENT_TIMESTAMP(3)"), server_onupdate=FetchedValue()
    )
    deleted_at: Mapped[datetime | None] = mapped_column(DATETIME(fsp=3))
```

SQL 과 비교 (앞에 없던 것만)

SQL (`01_phase1_account.sql`)	파이썬
`bpm DECIMAL(6,2) NOT NULL DEFAULT 120.00`	`Mapped[Decimal]` · `Numeric(6, 2)` · `server_default=text("120.00")`
`time_sig_num TINYINT UNSIGNED NOT NULL DEFAULT 4`	`Mapped[int]` · `TINYINT(unsigned=True)`
`master_settings JSON NULL`	`Mapped[dict[str, Any] | None]` · `JSON`
`template_id BIGINT UNSIGNED NULL` + (Phase 2 에서 `ALTER` 로 FK 추가)	`Mapped[int | None]` · **FK 는 적지 않음** — 아래 §5

문법 풀이

import 문법	뜻
`from 모듈 import 이름`	모듈 안의 `이름` 만 가져와 그대로 씀
`from sqlalchemy.dialects.mysql import …`	SQLAlchemy 안의 MySQL 전용 타입

식	뜻
`Numeric(6, 2)`	전체 6자리, 소수점 아래 2자리 → 최대 `9999.99`
`Decimal("98.50")`	소수를 정확하게 저장. `float` 의 `0.1 + 0.2 = 0.30000000000000004` 같은 오차가 없음
`TINYINT(unsigned=True)`	0 ~ 255 정수
`dict[str, Any]`	열쇠는 글자, 값은 아무 타입인 사전. JSON 객체와 같은 모양
`mapped_column(JSON)`	저장할 때 사전 → JSON 글자, 읽을 때 JSON 글자 → 사전으로 자동으로 바꿈

4. 흐름 (새 프로젝트 하나를 넣을 때)

1. `Project(owner_id=7, title="My Song")` — 2칸만 넣는다.
2. `db.add` → `db.flush()` 로 `INSERT`.
3. DB 가 `project_id` · `bpm=120.00` · 박자 `4/4` · `sample_rate=48000` · `revision=0` · `created_at` 을 채운다.
4. P1-13 에서는 같은 저장 안에서 OWNER 멤버(11)와 MASTER 트랙(12)도 함께 넣는다 (FR-122).

5. 왜 이렇게

선택	이유
`bpm` 은 `Decimal`	화면에 `98.50` 을 넣으면 `98.50` 으로 돌아와야 한다. 입력 스키마(`schemas/project.py`)도 `Decimal` 이라 그대로 넘긴다
`template_id` 에 FK 를 적지 않음	가리키는 `project_templates` 모델이 아직 없다(Phase 2). 없는 테이블을 가리키면 SQLAlchemy 가 저장할 때 오류를 낸다. 실제 FK 는 DB 에 있으므로 잘못된 번호는 DB 가 막는다
`revision` 은 그냥 칸으로	저장할 때마다 +1 해서 동시 저장 충돌을 찾는 데 쓴다. 어떻게 올릴지는 저장 기능(Phase 2)에서 정한다
관계는 다음에	`project.members` · `project.tracks` 는 `ProjectMember`(11) · `Track`(12) 칸이 채워져야 붙일 수 있다
색인(`idx_projects_owner` · `idx_projects_updated`)은 적지 않음	08 · 09 와 같은 이유 — DB 의 검색용

6. 시험

`tests/test_project_model.py` (새로 만듦 · 4개) — 실제 MySQL. 넣은 행은 끝나면 되돌린다 (시험 뒤 확인: 0행).

시험	확인한 것
칸이 DB 와 같은지	칸 17개 이름 · 순서 · `NULL` 허용 · FK(`owner_id` → `users`)
2칸만 넣기	`bpm` `120.00` · 박자 `4/4` · `48000` · `revision 0` · `master_settings` 비어 있음 · `created_at` 채워짐

시험	확인한 것
값을 넣고 DB 에서 다시 읽기	`bpm 98.50` · 박자 `3` · JSON `{"gain_db": -1.5, "limiter": {"on": true}}` 가 그대로
없는 회원 번호를 `owner_id` 로	DB 가 거절 (`IntegrityError`)

`tests/conftest.py` 의 `assert_matches_db` 보완

바꾼 것	이유
FK 의 `ON DELETE` 도 비교	11 `ProjectMember` 의 `ON DELETE CASCADE` 가 DB 와 같은지 보려고
아직 모델이 없는 테이블을 가리키는 FK 는 비교에서 뺌	`template_id → project_templates` (Phase 2)

전체 시험 61개 통과, `ruff` 검사 통과.